Pump Up The Jam

Analog Keyboard Synthesizer

By: Zach Schultz & Jack Haggard

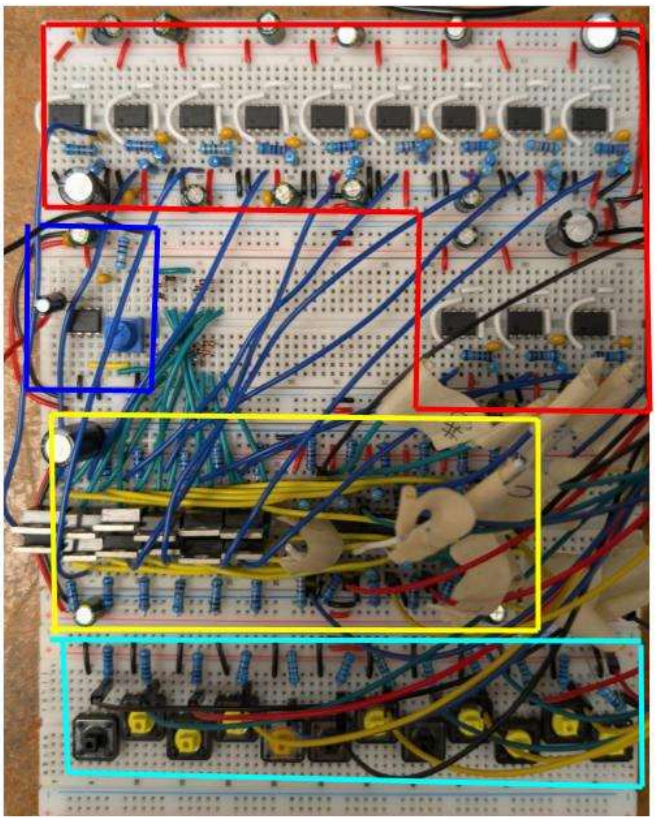
Final Project Report for ECEN 5613

Introduction (Jack)

Pump Up The Jam is an analog synthesizer with variable volume control and note control. The microcontroller takes input from the keyboard and translates that into an output which controls which tones are played. Additionally, the project implements an LCD to display what notes are currently being played.

Technical Description

Board Design



Overview of the board:
Red: signal generators
Blue: speaker amplifier
Yellow: signal enablers
Cyan: keyboard buttons

New Hardware

The new hardware that makes up Pump Up The Jam is composed of three sections: the signal generators, the signal enablers, and the speaker amplifier. All three of these components are necessary for playing one or more notes at a time.

The signal generators used are 555 timers oriented to output a square wave with a 50% duty cycle (Figure 2). This will be the basis for all 12 notes that this project can create with each note needing its own 555 timer. The frequency was calculated with $f = \frac{1}{1.66RC}$ and then adjustments were made to the time constant if the frequency of the sound was not within ±3Hz of the target note.

Table 1: Table for resistor and capacitor values

Note	**Resistor**	**Frequency (C3-B3)**	**Capacitor**
C	30593.27707	130.8	0.00000015
C#	28871.57749	138.6	0.00000015
D	27258.85995	146.8	0.00000015
Eb	56729.17889	155.6	0.000000068
E	53562.25871	164.8	0.000000068
F	50555.90055	174.6	0.000000068
F#	69032.7885	185	0.000000047
G	65158.49935	196	0.000000047
G#	61488.03983	207.7	0.000000047
A	124016.5488	220	0.000000022
Bb	117046.9358	233.1	0.000000022
B	110504.8227	246.9	0.000000022

The signal enablers use active low logic coming from the STM32f429I. It requires a non-inverting amplifier to bump the 3.3V to 9V logic for the MOSFET gate (Figure 3). The gain is calculated to be 2.5 making the output voltage of the amplifier at ~9V. The source of the P-MOSFET is connected to the 555 timer outputting the square wave, the drain is connected to the speaker circuit, and the gate is tied to the output of the amplifier. Each note will need this circuit, totaling to 12 signal enablers.

For amplifying the signal to output into a speaker, we chose the LM386 amplifier with the default gain of 20 (Figure 1). A 10kΩ potentiometer is used as a volume knob with Vin being the combined signals coming from the 12 signal enablers.

Firmware Design (Jack)

Given that the creation and amplification of tones is handled in hardware the firmware is relatively simple. At its core is a polling while loop which checks the 12 input “keys” and updates their corresponding outputs. Additionally in each loop the display is updated to reflect what notes are currently being played. All inputs and outputs to the board are handled on the GPIO pins of the STM32F429i-Discovery. This specific STM dev board also implements a 240x320 LCD display.

The first step of the firmware loop checks the voltage state of 12 keys, where each key is configured as a button with a pull up resistor such that when the button is pressed the voltage goes low representing a logic high and when the button is unpressed the voltage is high representing a logic low. These 12 states are then captured by the STM32F and are used for 2 purposes: outputting to the mosfets the correct logic levels, and updating the LCD with the notes that are being played. The mosfets that control whether a note is or isn’t played operates on active low logic. Finally the LCD is implemented through the use of the HAL and code that both group members previously implemented in past course work dealing with the same STM32F dev board allowing for the implementation to be made far simpler.

Testing Process

The first testing to be done is measuring a consistent frequency from the 555 timer circuit (Figure 2). The circuit was powered with 9 volts and then the frequency was measured with an oscilloscope. We aimed for a frequency within ±3Hz of the target frequency.

The next test was to measure the frequency of the sound coming from the speaker. Once the amplifier circuit (Figure 1) is set up, a tuner is used to measure the frequency of the sound coming from the speaker. This is a necessary test since oftentimes the frequency of the 555 timer and the frequency of the sound do not match up due to issues with the solderless breadboard.

Next was testing the combination of multiple frequencies. By placing two diodes with the cathodes facing the input of the amplifier and the anodes connecting to the outputs of the 555 timers, multiple frequencies can be played from the same amplifier.

The second to last test is making sure the MOSFETs can turn the frequencies on and off when instructed. The source is connected to the output of the 555 timer, the drain is connected to the anode of the diode, and the gate will either be tied to voltage high or voltage low. Voltage high should turn off the signal and voltage low should turn on the signal. This test should also incorporate playing multiple signals at once.

The last test to make sure the 3.3V logic gets amplified to 9V logic from the STM32 to the hardware (Figure 3). Using a non-inverting amplifier, step the 3.3V logic to 9V logic so that the gate of the MOSFETs can open and close properly. Just as the test before, high voltage will turn off the signal and low voltage will turn on the signal.

Results and Error Analysis

Pump Up The Jam revision one was a success. It produced sound and played multiple notes at once. It also displayed which note was being displayed on the screen and can be coded to play certain melodies. However there are still some current errors that need to be addressed.

The notes that are played are inconsistent. Sometimes it will play the correctly calculated frequency then it will switch to a completely different note. We believe this is due to signal integrity with the solderless breadboard. This issue may be fixed if the project was instead soldered onto a perf board.

Another issue is there is a quiet white noise that appears when none of the notes are activated. This error is an error that could be from multiple different sources, such as that all signals share one line, or that the breadboard is causing interference, or some other unknown reason.

Conclusion

This project implemented a common passion and interest between both group members and did so with little stress and a great deal of fun. While individually each of the components of this project were used by one or both of us extensively in past projects and coursework, integrating all of it together has proven to be an interesting exercise in debugging and integration. This project has also laid the groundwork for considerable further improvements, a product both of the integration process and the continued passion and drive to refine this project.

Future Development Ideas

Next steps for this project would be to have more accurate output frequencies. Some of the notes seem to fail when toggled on for a long period of time and can affect the sound. We would also like to find a better method for mixing the signals. Although multiple notes can be played at the same time, since they are just square waves they clash with each other therefore the notes do not mesh together well. This may have us implementing a different filtering method, a different mixing method, or maybe some other choice we have not discovered.

Something else that would be interesting to add would be a switch to toggle the type of wave output. Something to output a sine wave, a square wave, and a triangle wave would add a lot of depth to this project.

Acknowledgements

Steven Dunbar - inspired the use of 555 timers
Zane McMorris - debugging help

References

Inspiration for signal mixing: https://beavisaudio.com/projects/cmossynthesizers/

Appendix

A1 - Bill of Materials

- 555 Timer - 12
- Push Button - 12
- P-type MOSFET - 12
- T272 Op-Amp - 6
- STM32f429I-DISC1
- LM386 Amplifier
- 10kΩ Potentiometer
- Various Resistors
- Various Capacitors
- IN4148 Diode - 12
- Wire
- Solderless Breadboard - 4

A2 - Schematics

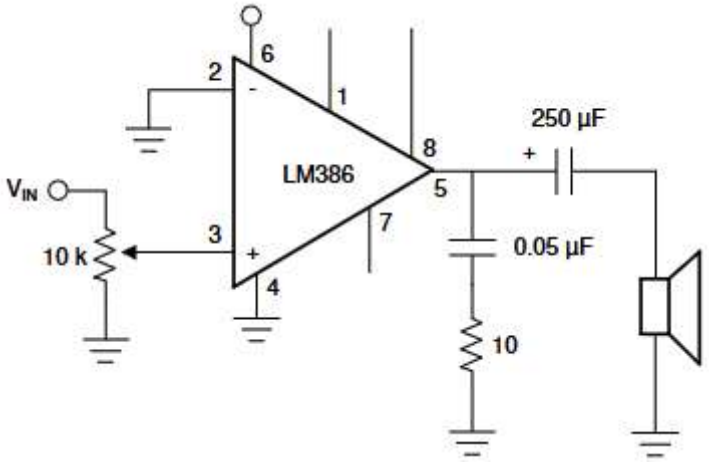


Figure 1: Speaker amplifier circuit

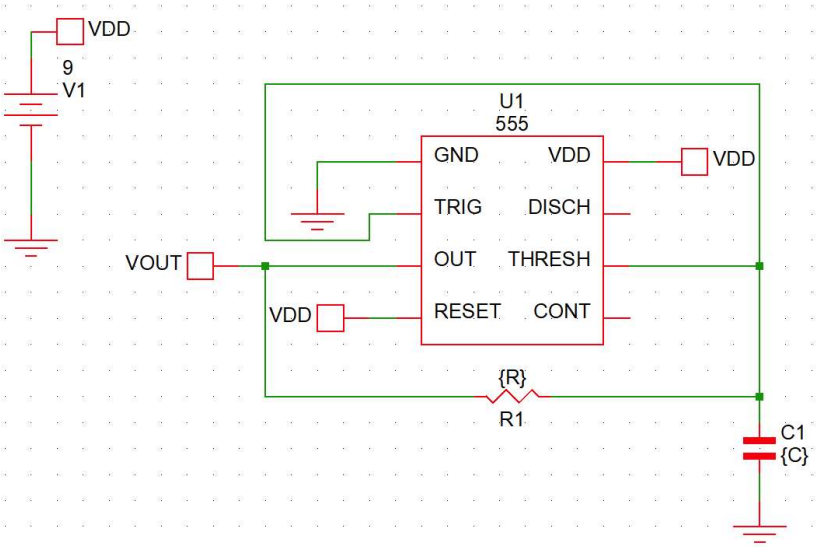


Figure 2: 555 timer circuit, creates a 50% duty cycle square wave at VOUT

$$f = \frac{1}{1.66RC}$$

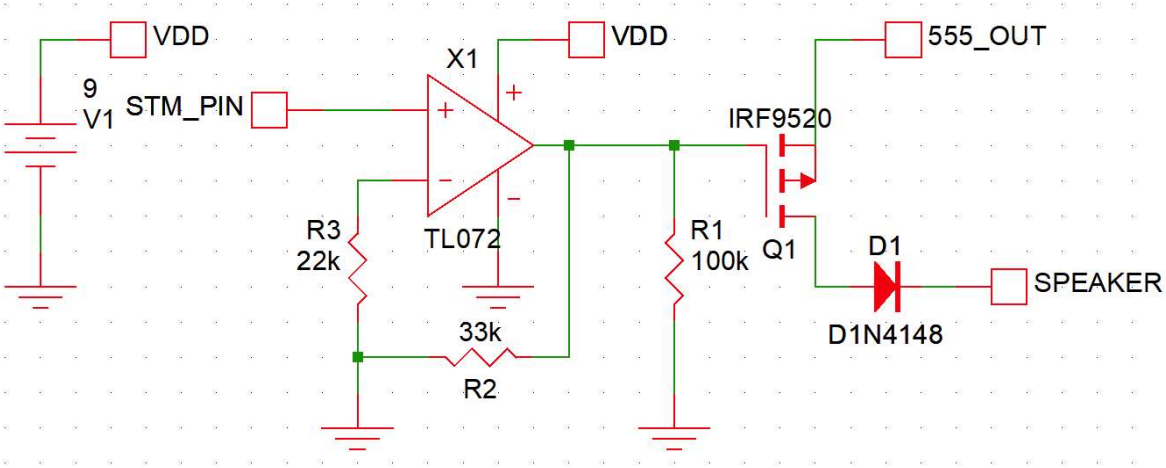


Figure 3: Logic and 3.3V amplifier circuit, gain is 2.5

A3 - Firmware Source Code (Jack)